

PARALLEL PROCESSING

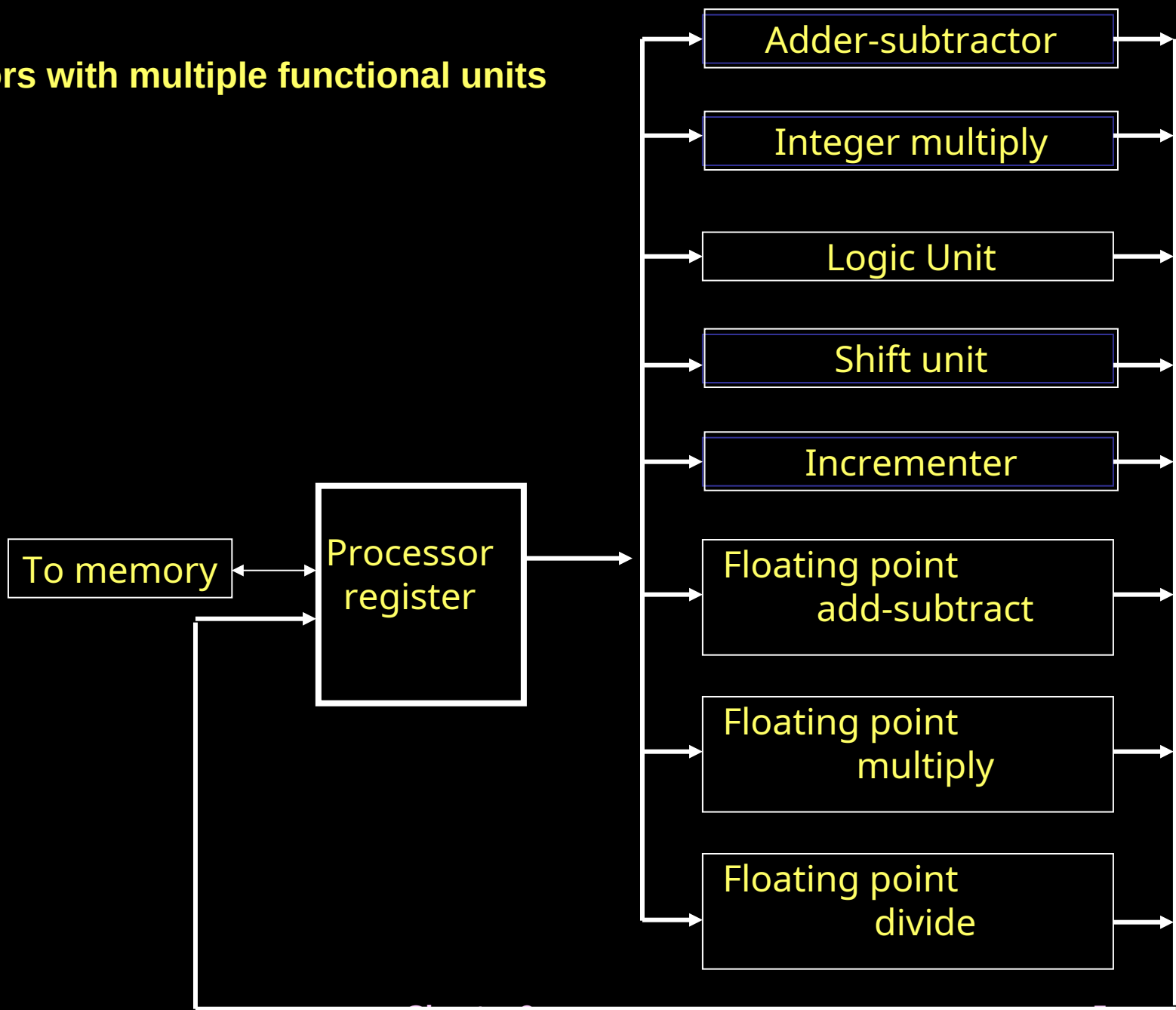
- **Parallel processing** is a term used to denote a techniques that is used to provide simultaneous data-processing task for the purpose of increasing the computational speed of a computer.

Parallel Processing with multiple functional units

- ✦ Parallel processing is established by distributing the data among the multiple functional units.
- ✦ For eg. the arithmetic, logic, and shift operations can be separated into three units and the operands are diverted to each unit under the supervision of CU.

★ The fig. below shows one way of separating the execution unit into eight functional units operating in parallel.

Processors with multiple functional units



- ★ **The operands are applied to one of the units depending on the operation specified by the instruction associated with the operands.**
- ★ **The operation performed in each functional unit is shown in each block.**

- ★ **The adder and integer multiplier performs the arithmetic operations with integer numbers.**
- ★ **The floating-point operations are separated into three circuits operating in parallel.**

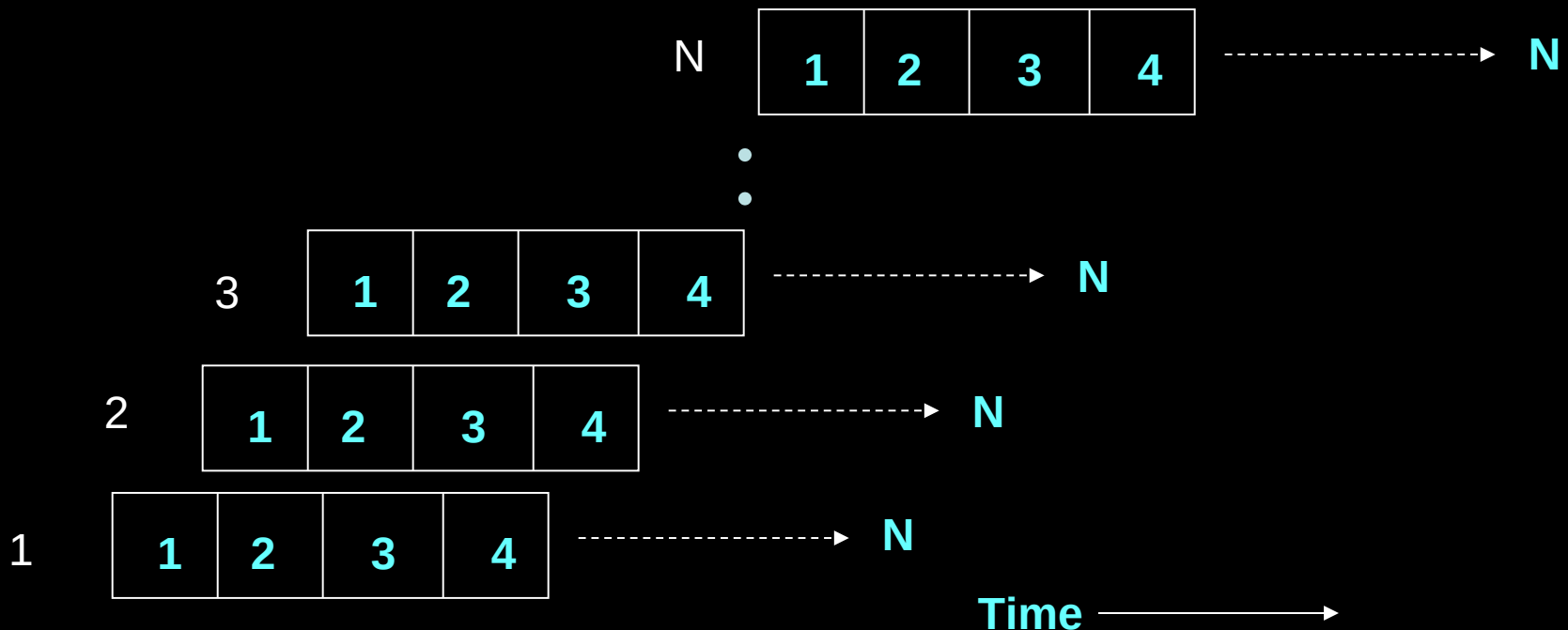
- ★ **The logic, shift and increment operations can be performed concurrently on different data.**
- ★ **All units are independent of each other, so one number can be shifted while another number is being incremented.**
- ★ **The CU coordinates all these multiple functional units.**

Pipelining

- ✦ Break down a sequential process into suboperations.
- ✦ Each subprocess executes in a dedicated segment
- ✦ All segments execute simultaneously
- ✦ Binary data flows from one segment to the next one during a clock pulse
- ✦ Each segment is a combinational circuit with its result stored in a register @ clock pulse

- **Pipelining is a technique used in advanced microprocessors where the microprocessor begins executing a second instruction before the first has been completed.**
- **That is, several instructions are in the pipeline simultaneously, each at a different processing stage.**
- **The fig below shows a continuous stream of jobs going through N sequential steps of process.**

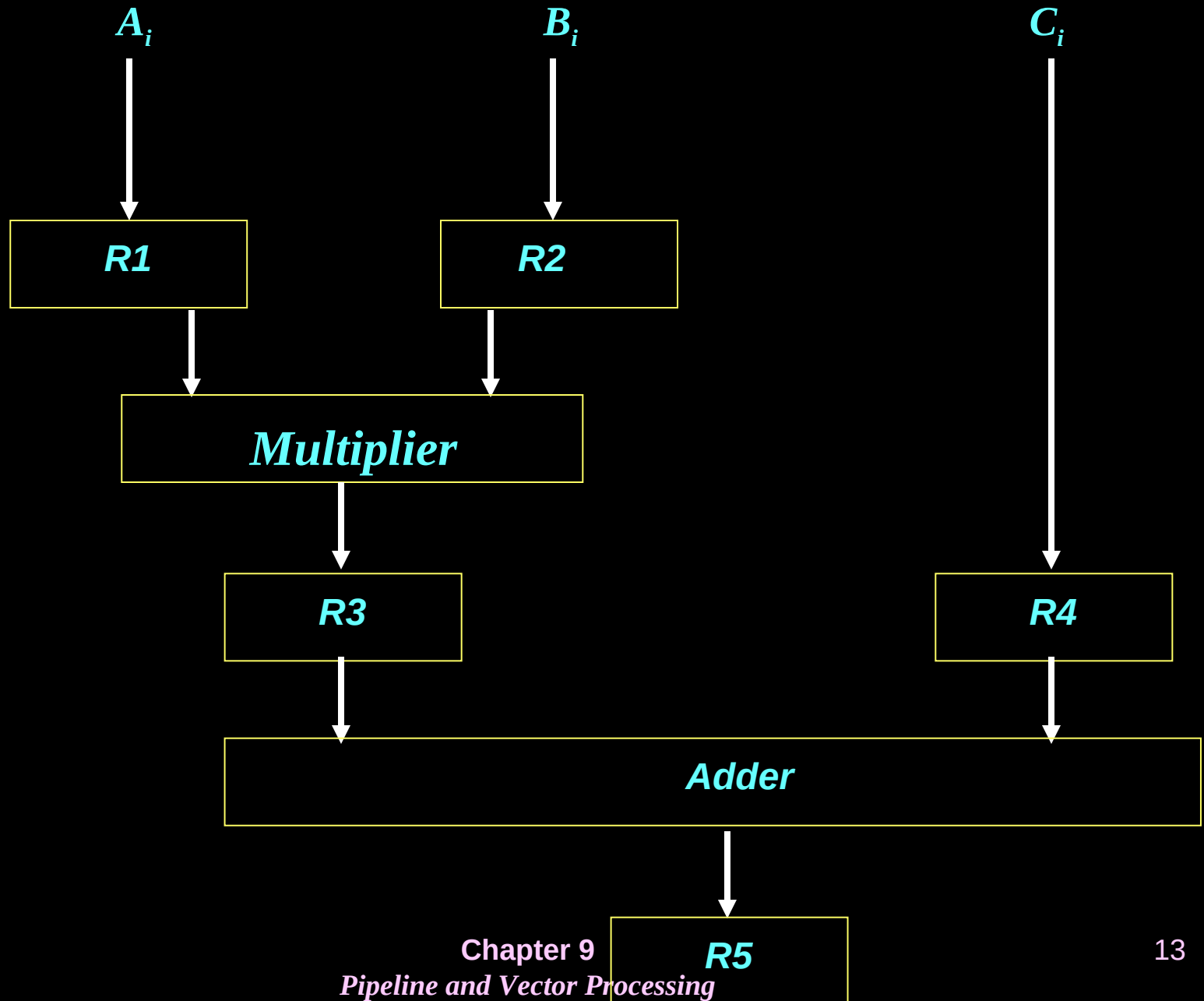
★ The fig below shows a pipelined execution of an N step process.



Pipeline processing

- ★ The simplest way of viewing the pipeline structure is to imagine that each segment consist of an input register followed by a combinational circuit.
- ★ The register holds the data and the combinational circuit performs the sub-operation in the particular segment.

Example of a Pipeline processing



- ★ **The output of the combinational circuit in a given segment is applied to the input register of the next segment.**
- ★ **A clock is applied to all registers to perform all segment activity.**
- ★ **In this way the information flows through the pipeline one step at a time.**

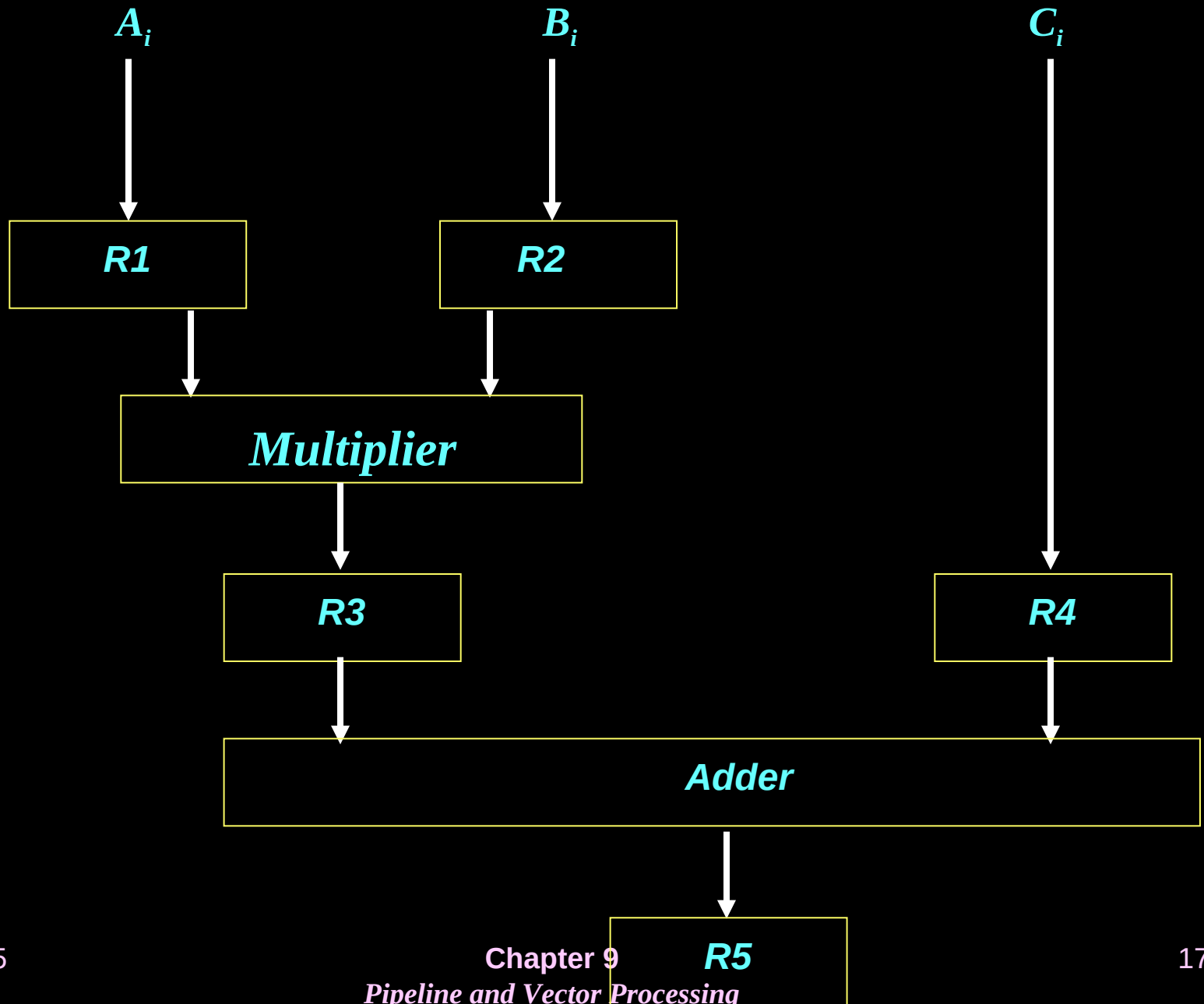
- ★ Pipeline organization can be explained by means of an example. Suppose we want to perform the combined multiply and add operations with a stream of numbers.

$$A_i * B_i + C_i \text{ for } i=1,2,3, \dots, 7$$

- ★ Each suboperation is to be implemented in a segment within a pipeline.

- ★ Each segment has one or two registers and a combinational circuit as shown in the figure below:

Example of a Pipeline processing



- ★ **R1 through R5 are registers that receives new data with every clock pulse.**
- ★ **The multiplier and adder are the combinational circuits.**

★ The suboperations performed in each segment of the pipeline are as follows:

$R1 \leftarrow A_i, \quad R2 \leftarrow B_i$ Input A_i and B_i

$R3 \leftarrow R1 * R2, \quad R4 \leftarrow C_i$ Multiply and
input C_i

$R5 \leftarrow R3 + R4$ Add C_i to product

- ★ The five registers are loaded with new data at every clock pulse.
- ★ The effect of each clock is shown in the table below:

Clock Pulse Number	Segment 1		Segment 2		Segment 3
	<i>R1</i>	<i>R2</i>	<i>R3</i>	<i>R4</i>	<i>R5</i>
1	A_1	B_1	—	—	—
2	A_2	B_2	$A_1 * B_1$	C_1	—
3	A_3	B_3	$A_2 * B_2$	C_2	$A_1 * B_1 + C_1$
4	A_4	B_4	$A_3 * B_3$	C_3	$A_2 * B_2 + C_2$
5	A_5	B_5	$A_4 * B_4$	C_4	$A_3 * B_3 + C_3$
6	A_6	B_6	$A_5 * B_5$	C_5	$A_4 * B_4 + C_4$
7	A_7	B_7	$A_6 * B_6$	C_6	$A_5 * B_5 + C_5$
8	—	—	$A_7 * B_7$	C_7	$A_6 * B_6 + C_6$
9	—	—	—	—	$A_7 * B_7 + C_7$

- ★ The **first clock** pulse transfers A1 and B1 into R1 and R2.
- ★ The **second clock** pulse transfers the product of R1 and R2 into R3 and C1 into R4.
- ★ The same clock pulse transfers A2 and B2 into R1 and R2.

- ★ The **third clock** pulse operates on all three segments simultaneously.
- ★ It places A3 and B3 into R1 and R2, transfers the product of R1 and R2 into R3, transfers C2 into R4, and places the sum of R3 and R4 into R5.
- ★ It takes three clock pulses to fill up the pipe and retrieve the first output from R5.

Normalized and Unnormalized forms of floating point numbers

- ✦ A floating point is said to be normalized if the most significant position of the mantissa contains a non-zero digit.
- ✦ For e.g. the mantissa of a decimal number 0264 is not normalized where as 2640 is normalized.
- ✦ When the mantissa is normalized, it has no leading zeros and hence contain the maximum number of significant digits.

★ For e.g.. The decimal number 3456.892 is represented in the two forms as follows:

Normalized form : .3456892 * 10⁴

Unnormalized form : .03456892 * 10⁵

Arithmetic Pipeline

- ★ Arithmetic Pipeline units are found in very high speed computers.
- ★ They are used to implement floating-point operations, multiplication of fixed-point numbers and similar other computations in scientific problems.

★ The inputs to the floating-point adder are two normalized floating-point binary numbers:

$$\square X = A \times 2^a$$

$$\square Y = B \times 2^b$$

★ A and B represent mantissa part and a and b are the exponents.

- ★ **The register labeled R is placed in between the segments to store intermediate results.**
- ★ **The floating point addition and subtraction can be performed in four segment as shown below:**

Exponents

a

b

Mantissas

A

B

Segment 1:

Segment 2:

Segment 3:

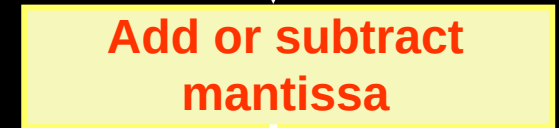
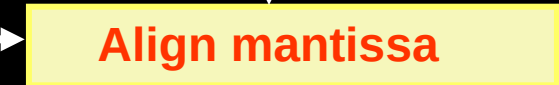
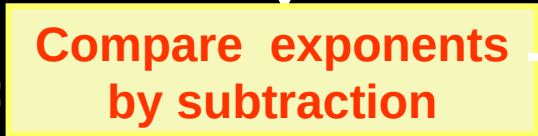
Segment 4:

12/08/25

Chapter 9

Pipeline and Vector Processing

29



Difference

Normalized results

★ The suboperations that are performed in the four segments are:

1. Compare the exponents.
2. Align the mantissa.
3. Add or subtract the mantissas.
4. Normalize the result.

- ✦ **Consider two normalized floating point numbers:**

$$X = 0.9504 \times 10^3$$

$$Y = 0.8200 \times 10^2$$

- ✦ **In the first segment, compare the two exponents, if the exponents are not same then find the difference. The two exponent are subtracted in the first segment to obtain $3 - 2 = 1$.**
- ✦ **The larger exponent 3 is chosen as the exponent of the result.**

★ The exponent difference determines how many times the mantissa associated with the smaller exponent must be shifted to the right. This produces the alignment of the two mantissas.

★ The second segment shifts the mantissa of Y to the right to obtain

$$X = 0.9504 \times 10^3$$

$$Y = 0.0820 \times 10^3$$

This aligns the two mantissa under the same exponent.

- ★ In segment three, the addition of two mantissas produces the sum

$$Z = 1.0324 \times 10^3$$

- ★ The result is normalized in segment four.

- ★ The sum is adjusted by normalizing the result so that it has a fraction with a non-zero first digit.
- ★ This is done by shifting the mantissa once to the right and incrementing the exponent by one to obtain the normalized sum.

$$Z = 0.10324 \times 10^4$$

Summary of operation of Pipeline for floating-point addition and subtraction

$$X = 0.9504 \times 10^3$$

$$Y = 0.8200 \times 10^2$$

Subtract exponents: $3 - 2 = 1$

Align mantissas

$$X = 0.9504 \times 10^3$$

$$Y = 0.0820 \times 10^3$$

Add mantissas:

$$Z = 1.0324 \times 10^3$$

Normalize result:

$$Z = 0.10324 \times 10^4$$

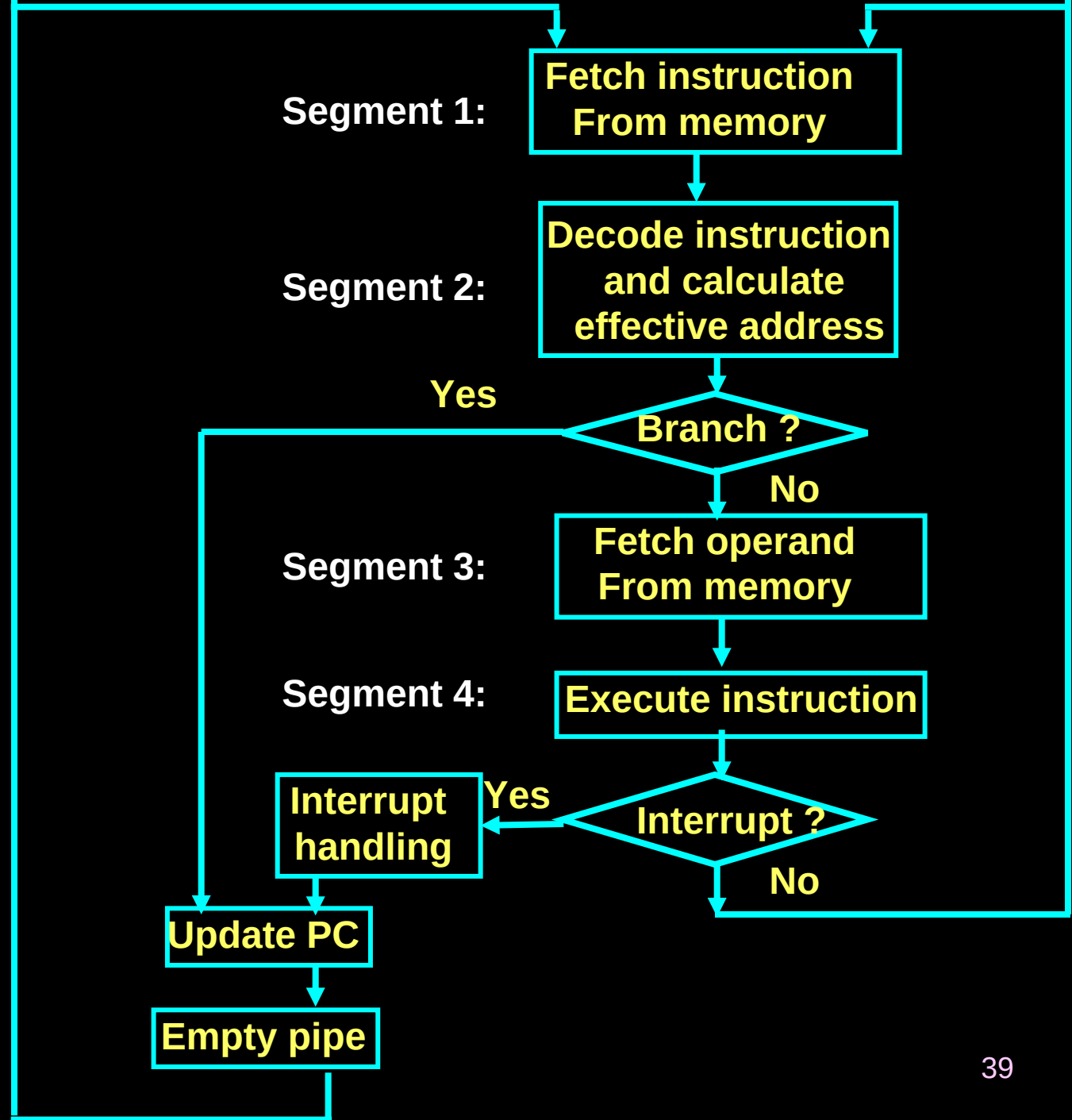
Instruction Pipeline

- ★ An instruction pipeline reads consecutive instructions from memory while previous instructions are being executed in other segments.

- ★ **There is one case which degrade the performance of the system when a branch instruction is executed.**
- ★ **In this case, all the instructions that were fetched have to be flushed out, and a new set of instructions are fetched.**

★ The fig below shows how the instruction cycle in the CPU can be processed in a four segment pipeline.

Four segment CPU pipeline



- ★ The figure below show the space-time diagram of instruction pipeline.
- ★ The time in the horizontal axis is divide into steps of equal duration.

- ★ **The four segments are represented in the diagram with an abbreviated symbol.**
 - **FI : instruction fetch segment**
 - **DA : decode and effective address calculation segment**
 - **FO : operand fetch segment**
 - **EX : execute segment**

SPACE TIME DIAGRAM

As soon as this instruction is decoded in segment DA in step 3, the transfer from FI to DA of other instruction is halted until the branch instruction is executed.

Step:	1	2	3	4	5	6	7	8	9	10	11	12	13
Instruction 1	FI	DA	FO	EX									
2		FI	DA	FO	EX								
(Branch) 3			FI	DA	FO	EX							
4				FI	-	-	FI	DA	FO	EX			
5								FI	DA	FO	EX		
6									FI	DA	FO	EX	
7										FI	DA	FO	EX

- ✦ It is assumed that the processor has separate instruction and data memories so that the operation in FI and FO can proceed at the same time.

- ✦ **In step 4,** the instruction 1 is being executed in segment EX;
- ✦ The operand for instruction 2 is being fetched in segment FO;
- ✦ Instruction 3 is being decoded in segment DA;
- ✦ And instruction 4 is being fetched from memory in segment FI.

- ✦ **Assume that instruction 3 is a branch instruction.**
- ✦ **As soon as this instruction is decoded in segment DA in step 3, the transfer from FI to DA of other instruction is halted until the branch instruction is executed.**
- ✦ **The pipeline then continues until a new branch instruction is encountered.**

Data Hazards

- ★ **Data Hazards** occur when an instruction depends on the result of previous instruction and that result of instruction has not yet been computed.
- ★ Whenever two different instructions use the same storage, the location must appear as if it is executed in sequential order.

- Data hazards can be classified as
- *Flow dependencies (Read-After-Write),*
- *Output dependencies (Write-After-Write)*
and
- *Anti-dependencies (Write-After-Read).*

Read-After-Write (RAW) Hazards

- ✦ A Read-After-Write hazard occurs when an instruction requires the result of a previously issued, but as yet uncompleted instruction
- ✦ RAW hazard occurs when instruction J tries to read data before instruction I writes it.
- ✦ RAW Example
- ✦ $R6 = R1 * R2$
 $R7 = R5 + R6$

- ✦ **Write-After-Write (WAW) Hazards**
- ✦ These hazards occur when the output register of an instruction is used for write after written by previous instruction.
- ✦ In the WAW example shown in the figure, both instructions write their results to R6.
- ✦ **WAW hazard occurs when instruction J tries to write output before instruction I writes it.**
- ✦ **WAW Example**
- ✦ **$R6 = R1 * R2$**
 $R6 = R4 + R5$

★ **Write-After-Read (WAR) Hazards**

★ These hazards occur when the output register of an instruction is used right after read by a previous instruction.

★ **WAR hazard occurs when instruction J tries to write data before instruction I reads it.**

★ **WAR Example**

★ $X3 = X1 / X2$
★ $X5 = X4 * X3$

What is an Embedded System?

- ✦ An embedded system is a microprocessor-based computer hardware system with software that is designed to perform a dedicated function, either as an independent system or as a part of a large system.
- ✦ At the core is an integrated circuit designed to carry out computation for real-time operations.

- They may function as standalone devices or as part of larger systems, hence the term "embedded,"
- Like most computers, embedded systems are a combination of hardware and software, usually:
- Microprocessors or microcontrollers
- Graphics processing units (GPUs)
- Volatile and non-volatile memory
- Input/output communication interfaces and ports

Different types of Embedded systems

★ When classifying embedded systems based on performance and functional requirements, embedded systems are divided into four categories:

1. Real-time embedded systems
2. Standalone embedded systems
3. Network, or networked, embedded systems
4. Mobile embedded systems

- Real-time embedded systems must provide results or outputs promptly.
- Priority is assigned to output generation speed, as real-time embedded systems are often used in mission-critical sectors, such as defense and aerospace, that need important data.
- Examples of real-time embedded systems include:

- ✦ Aircraft controls
- ✦ Land-vehicle and flight computers that process and transmit sensor-acquired data
- ✦ Missile defense system controls

standalone embedded systems

- Standalone embedded systems don't require a host computer to function. They can produce outputs independently.
- Examples of standalone embedded systems include:
- Digital cameras
- Digital wristwatches
- MP3 players
- Appliances, such as refrigerators, washing machines, and microwave ovens

Networked embedded systems

- ✦ Networked, embedded systems rely on wired or wireless networks and communication with web servers for output generation.
- ✦ Frequently cited examples of network embedded systems include:
- ✦ Home and office security systems
- ✦ Automated teller machines (ATMs)

Mobile embedded systems?

- Mobile embedded systems refer specifically to small, portable embedded devices, such as cellphones, laptops, and calculators.
- All mobile embedded systems are standalone embedded systems, but not all standalone embedded systems are mobile embedded systems.

END